A Unified Sizing Model for User Interfaces

Huu Khanh Nguyen

***Abstract*—Sizing in modern design systems relies on ad-hoc token tables and per-element overrides, producing inconsistent spatial relationships that degrade both visual coherence and accessibility. This paper presents a unified sizing model that derives all element geometry — height, padding, and border radius — from three variables: intrinsic text lines n, wrapping level w, and density factor d, expressed in a single base unit U = fontSize$/4$. The model introduces a four-level wrapping taxonomy that classifies every UI element from inline text to structural overlays, and a five-step density scale whose values reproduce the canonical button heights adopted independently by major design systems. A dimension table mapping 69 production patches to the model confirms full coverage without exceptions. By reducing spatial design to closed-form expressions over (n, w, d), the framework eliminates per-element magic numbers while preserving the proportional relationships that practitioners have converged on through iteration.**

Keywords: sizing model; design systems; element geometry; density; spacing; UI framework

This is the second of two papers from the Domphy project. The first paper [29] addresses the chromatic axis (color palette metrics). This paper addresses the spatial axis (sizing and density). Together they form a quantitative foundation for UI design systems.
Reference implementation and production validation: https://github.com/domphy/domphy

I. Introduction

Sizing is among the most fundamental decisions in interface design, yet it remains one of the least formalized. Current design systems approach element sizing through one of two strategies:

- **Token tables:** Predefined size tokens (e.g., `sm`, `md`, `lg`) map directly to pixel values. Each element maintains its own mapping, leading to divergent height and spacing values across the system.
- **Proportional scales:** Typographic scales provide font-size ratios, but spatial properties such as padding, height, and radius are still set independently per element.

Both approaches share a structural limitation: there is no single expression from which all spatial properties of an element can be derived. This leads to several practical consequences:

- Alignment between elements of different types requires manual coordination.
- Density changes (compact vs. comfortable layouts) require per-element overrides rather than a global parameter.
- Responsive rescaling breaks spatial proportions because height and padding scale by different rules.

This paper proposes a sizing model that reduces element geometry to three variables and one base unit. The model is designed with three goals:

1) Every element's height, padding, and radius should be expressible as a closed-form function of (n, w, d).
2) Changing the density factor d should uniformly affect all elements without per-element adjustment.
3) The default parameterization should reproduce the button heights that major design systems have independently converged on.

The remainder of the paper is organized as follows. Section III introduces the base unit, core variables, and the sizing pipeline. Section IV derives the geometry formulas from a minimal constraint set and argues for their minimality. Sections V–V-I present the formulas and their properties. Section VI provides a complete coverage proof over 69 production patches. Section VII benchmarks the model against industry design systems. Section VIII discusses the relationship to the Chromametry tone model. Section IX summarizes the contribution.

II. Related Work

A. Modular Scales and Spatial Systems

The idea that typographic dimensions should follow a mathematical ratio rather than ad-hoc pixel values was formalized by Bringhurst [1], who defined a modular scale as "a prearranged set of harmonious proportions." Brown [2] extended this concept to web design, demonstrating that a single ratio (e.g., the golden ratio $1{:}1.618$) can generate a full hierarchy of font sizes, line heights, and margins. However, modular scales address only the *typographic* axis; they do not prescribe how element geometry — height, padding, radius — should relate to the type scale. The present model bridges this gap by deriving all spatial properties from one base unit anchored to font size.

Modern design systems organize spatial values as *design tokens*: named key-value pairs that encode colors, sizes, and spacing [5]. Curtis [3] catalogued the spatial token taxonomy used in production systems (inset, stack, inline, grid) and showed that spacing is the most frequently referenced property category across UI libraries. Kutschmann [4] identified a structural limitation of flat token systems: tokens lack relational semantics, so changing one value (e.g., density) requires manually updating every dependent token. The parametric model proposed here addresses this limitation: all spatial tokens are derived from (n, w, d), so a single density change propagates automatically.

The 4 px / 8 px grid has become a de facto industry standard. Dahl [6] traced its adoption to high divisibility (halves,

quarters map to integer pixels at common device pixel ratios) and its endorsement by Apple Human Interface Guidelines and Google Material Design. The base unit $U = \text{fontSize}/4$ at the default font size of 16 px yields $U = 4\,\text{px}$, aligning the model with this grid without hardcoding it.

B. Density

Google Material Design 2 introduced a formal density scale for element sizing [7]. The scale defines integer density offsets $(0, -1, -2, -3)$, each decrementing element height by a fixed interval of $4\,\text{px}$:

$$h = h_{\text{base}} + 4\,\text{px} \times \delta,$$

where $\delta \leq 0$ is the density offset [8]. This approach demonstrates that density can be parameterized independently of type scale. However, the Material model applies density as an additive pixel offset rather than a multiplicative factor, which means the relationship between density and padding is implicit. The present model uses a multiplicative density factor d that enters directly into the padding and height formulas, making the relationship explicit and font-size-independent.

Information density as an ergonomic criterion has been studied in the HCI literature. Gabillon et al. [9] formalized density as a factor affecting perceptual and cognitive workload in composed interfaces. Tufte [10] introduced the data-ink ratio — the proportion of visual elements devoted to information versus decoration — which the density scale operationalizes for interactive UI by controlling the ratio of content height to surrounding whitespace.

C. Visual Consistency and Perceptual Grouping

The perceptual basis for consistent spacing derives from Gestalt grouping principles. Wertheimer [11] established that elements spaced closer together are perceived as related (the proximity principle). Wagemans et al. [12] provided a comprehensive modern review of grouping principles including proximity, similarity, common region, and uniform connectedness. The wrapping taxonomy introduced in Section V-A applies proximity at the element level: higher wrapping levels receive greater padding, creating a visual hierarchy that maps directly to structural containment.

The importance of consistent sizing for usability is well-established. Nielsen [13] identified consistency and standards as one of ten usability heuristics, arguing that consistent layout and sizing reduce cognitive load. Shneiderman et al. [14] formalized this as Rule 1 of interface design: "strive for consistency" in color, layout, fonts, and spacing. A unified sizing formula provides a mathematical guarantee of this consistency across element types.

D. Typography and Line Height

The model sets each text line at $6U = 1.5 \times \text{fontSize}$. WCAG 2.1 Success Criterion 1.4.12 [15] requires that content remains functional when users set line spacing to at least 1.5 times the font size. Adopting 1.5 as the default satisfies this resilience criterion by construction. The choice is independently supported by empirical evidence. Rello et al. [16] conducted an eye-tracking study with 104 participants showing that readability degrades at line-spacing extremes (0.8 and 1.8), with the 1.0–1.5 range yielding optimal performance. By baking the 1.5 ratio into the base content-height term, the model ensures WCAG compliance by construction.

E. Industry Button Heights

The height formula's default parameterization produces values that coincide with canonical button sizes across major design systems. At $\text{fontSize} = 16\,\text{px}$, $n = 1$, $w = 1$:

- MUI [17]: small $\approx$ 31 px, medium $\approx$ 37 px, large $\approx$ 42 px. MUI's fractional heights result from using $\text{lineHeight} = 1.75$ at $\text{fontSize} = 14\,\text{px}$.
- Ant Design [18]: default 32 px, large 40 px, derived proportionally from a single seed token.
- Chakra UI v2 [19]: sizes at $24, 32, 40, 48\,\text{px}$ — a linear $+8\,\text{px}$ progression. (Chakra v3 revised these to $24, 36, 40, 44, 48\,\text{px}$; the benchmark in Section VII uses v3 data.)
- GitHub Primer [20]: small 28 px, medium 32 px, large 40 px.
- Adobe Spectrum [21]: medium-scale element height 32 px, large/touch 40 px, built on a 4 px spacing base.

All systems use multiples of 4 px, and most default to 32 px for medium controls — consistent with $d = 1$ in the present model: $h = (6 + 2) \times 4 = 32\,\text{px}$. The convergence across independently developed systems suggests that the proportional relationship captured by the formula reflects a structural invariant rather than a coincidence.

III. Definitions

A. Base Unit

The sizing model is anchored to a single base unit derived from the local font size:

$$U = \frac{\text{fontSize}}{4}. \tag{1}$$

At the default body font size of 16 px, $U = 4\,\text{px}$. This value is not arbitrary: it produces integer pixel values at common density levels and aligns with the 4 px grid used by most production design systems [6].

When the local subtree inherits a different font size (via the size scale), U changes proportionally, and all formulas built on U rescale automatically.

B. Core Variables

Three variables parameterize all element geometry:

- $n \in \mathbb{N}^+$: the number of *intrinsic text lines* inside the element. For a single-line button, $n = 1$. For a text block displaying k lines, $n = k$.
- $w \in \{0, 1, 2, 3\}$: the *wrapping level*, a structural classification of the element's boundary type (see Section V-A).
- $d \in \mathbb{R}^+$: the *density factor*, controlling the compactness of the surrounding geometry without altering the type scale (see Section V-B).

C. Sizing Pipeline

The model operates as a three-stage pipeline:

1) **Scale resolution:** The local font size is resolved from an inherited size index, defining U.
2) **Geometry computation:** Formulas over (n, w, d) produce numeric spacing values in units of U.
3) **CSS emission:** Numeric results are converted to CSS length values via a spacing function.

This separation ensures that changing the type scale (stage 1) or the density (stage 2) does not require modifying the geometry formulas themselves.

IV. Derivation from Constraints

This section shows that the height formula $h = (n \cdot 6 + 2 \cdot d \cdot w) \cdot U$ is the simplest closed-form expression satisfying a minimal set of design constraints. Rather than being chosen among alternatives, the formula is the logical consequence of these constraints under a linearity assumption.

A. Constraint Set

C1. Line height = 1.5× font size. WCAG 2.1 Success Criterion 1.4.12 [15] requires that content remains functional when the user sets line spacing to at least 1.5 times the font size. Adopting 1.5 as the default line-height satisfies this resilience criterion by construction: no reflow occurs because the default already meets the threshold. This choice is independently supported by eye-tracking evidence showing that readability is optimal in the 1.0–1.5 range [16]. With $U = \text{fontSize}/4$, one text line occupies $1.5 \times \text{fontSize} = 6\,U$.

C2. Content height scales linearly with text lines. An element containing n text lines has content height $n \times 6\,U$. This follows from C1 by induction: if each line is $6\,U$, then n lines are $6n\,U$.

C3. Padding is symmetric. Block padding is applied equally above and below the content. Total vertical padding is $2 \times p_{\text{block}}$.

C4. Block padding is bilinear in density and wrapping level. Block padding must satisfy three requirements: (a) it increases linearly with density d, so that a compact layout has proportionally less padding; (b) it increases linearly with wrapping level w, so that structurally deeper containers have proportionally more padding; (c) it is expressed in units of U to preserve resolution independence. The unique bilinear form satisfying (a), (b), and (c) with unit coefficient is $p_{\text{block}} = d \cdot w \cdot U$.

C5. Inline elements ($w = 0$) have zero block padding. Elements without a boundary (plain text, icons, inline labels) have no vertical padding. Setting $w = 0$ in C4 yields $p_{\text{block}} = 0$, which satisfies this constraint automatically.

B. Height Derivation

Height is the sum of content height and total vertical padding:

$$\begin{aligned} h &= \underbrace{n \cdot 6\,U}_{\text{content (C1, C2)}} + \underbrace{2 \cdot p_{\text{block}}}_{\text{padding (C3)}} \\ &= n \cdot 6\,U + 2 \cdot d \cdot w \cdot U \quad \text{(C4)} \\ &= (n \cdot 6 + 2 \cdot d \cdot w) \cdot U. \end{aligned} \tag{2}$$

At $w = 0$, this reduces to $h = 6n\,U$ (C5). The formula is fully determined by the five constraints; no free parameters remain.

C. Padding Derivation

Block padding follows directly from C4:

$$p_{\text{block}} = d \cdot w \cdot U. \tag{3}$$

Inline padding requires an additional constraint:

C6. Inline padding maintains a target aspect ratio. For bounded controls, the horizontal padding should be wider than the vertical padding to accommodate text content. The ratio $p_{\text{inline}}/p_{\text{block}}$ should be approximately 3:1 for single-line controls ($w = 1$), decreasing for wider containers. The expression $\lceil 3/w \rceil$ produces this ratio:

w	$\lceil 3/w \rceil$	Ratio $p_{\text{inline}} : p_{\text{block}}$
1	3	3 : 1
2	2	2 : 1
3	1	1 : 1

This yields:

$$p_{\text{inline}} = \lceil 3/w \rceil \cdot d \cdot w \cdot U, \quad w \geq 1. \tag{4}$$

The ceiling function ensures integer ratios at each wrapping level. At $w = 3$, inline and block padding are equal, reflecting the fact that structural sections frame content symmetrically.

For bounded inline elements at $w = 0$ (e.g., tag, badge, inline code), block padding is zero (C5), but a small horizontal padding is needed to separate content from the boundary edge. This is governed by an additional constraint:

C7. Bounded inline padding is proportional to density. At $w = 0$, bounded inline elements receive $p_{\text{inline}} = 2d\,U$, providing horizontal spacing without vertical padding. Unbounded inline content (plain text, icons) has zero padding. This is a piecewise extension outside the $w \geq 1$ formula.

D. Radius Derivation

C8. Border radius equals block padding. Corner curvature should match the surrounding whitespace so that the boundary arc fits within the padding region. This produces:

$$r = p_{\text{block}} = d \cdot w \cdot U. \tag{5}$$

At $w = 0$, radius is zero (inline elements have no rounded corners). At higher wrapping levels, radius grows proportionally with padding, maintaining consistent visual weight.

E. Density Scale Derivation

The density values $D = [0.75, 1, 1.5, 2, 2.5]$ are not arbitrary. They are the values of d that produce integer or half-integer multiples of U in the single-line control case ($n = 1$, $w = 1$):

d	$p_{\text{block}} = d \cdot U$	$h = (6 + 2d) \cdot U$	Height (px)
0.75	$0.75\,U$	$7.5\,U$	30
1	$1\,U$	$8\,U$	32
1.5	$1.5\,U$	$9\,U$	36
2	$2\,U$	$10\,U$	40
2.5	$2.5\,U$	$11\,U$	44

At $U = 4\,\text{px}$, every density level produces an integer pixel height that is a multiple of 2. The reference implementation uses $d = 1.5$ as its default, yielding $h = 9\,U = 36\,\text{px}$. The industry benchmark (Section VII) shows that most design systems converge on $32\,\text{px}$ ($d = 1$) as their default medium height. The model accommodates both: the density scale is a parameter, not a prescription, and the choice of default d is a design decision independent of the formula itself.

F. Minimality

Given the bilinearity requirement in C4, the formula has no free parameters: every coefficient is determined by a named constraint. Alternative formulations must either:

- use a different line-height ratio (violating C1),
- use asymmetric padding (violating C3),
- use a non-linear relationship between padding and (d, w) (violating C4), or
- introduce additional parameters not determined by the constraints.

The last case would add degrees of freedom requiring ad-hoc choices. The present formulation is minimal in the sense that it uses the fewest assumptions (eight constraints) to fully determine all spatial properties, with no tunable parameters remaining.

V. Geometry Properties

The formulas derived in Section IV are restated here with their practical implications. All quantities are in units of U.

A. Wrapping Level

The wrapping level w classifies every UI element into one of four structural categories based on its boundary semantics:

w	Boundary class
0	Inline / no boundary
1	Single-line bounded control
2	Multi-line bounded block
3	Structural section / large overlay

Representative examples for each level:

- $w = 0$: plain text, icon, inline label. No outer boundary.
- $w = 1$: button, text input, select, tooltip. A single-line container with visible or implicit edges.
- $w = 2$: textarea, blockquote, card. A multi-line region with a defined boundary.
- $w = 3$: dialog, drawer, fieldset. A structural overlay or section that frames nested content.

The wrapping level is an intrinsic property of the element's role, not a runtime parameter. It is assigned once per patch type and does not change with density or font size.

This classification is exhaustive: every element in the UI library maps to exactly one wrapping level. Section VI confirms this by tabulating all production patches.

B. Density

The density factor d controls the spatial generosity of element geometry without altering the typographic scale. It acts as a multiplier in all padding and height formulas.

The model defines a five-step density scale:

$$D = [0.75,\ 1,\ 1.5,\ 2,\ 2.5]. \tag{6}$$

The reference implementation uses $d = 1.5$ (index 2) as its default. The industry benchmark (Section VII) shows that most systems converge on $d = 1$ ($h = 32\,\text{px}$) as their medium default. Both values are valid parameterizations of the same formula; the choice of default d is a design decision, not a property of the model.

Key properties of the density scale:

- **Separation from type scale:** Changing d modifies padding and height but leaves font size and line height unchanged. A compact layout ($d = 0.75$) and a comfortable layout ($d = 2.5$) share the same text rendering.
- **Global effect:** Density is inherited through the element tree via a `dataDensity` attribute. Setting it on a container adjusts all descendant elements uniformly.
- **Continuous multiplier:** Although the scale provides five discrete steps, d enters formulas as a real-valued multiplier, so intermediate values are algebraically valid.

The density factor participates in geometry as follows: it multiplies the wrapping-dependent padding contribution. At $w = 0$ (no boundary), d has no effect on height, because there is no padding to scale. At $w \geq 1$, increasing d adds symmetric block padding and therefore increases element height.

C. Height Formula

The height of an element is the sum of its content height and its vertical padding:

$$h = (n \cdot 6 + 2 \cdot d \cdot w)\, U. \tag{7}$$

The term $n \cdot 6\,U$ represents the intrinsic content height. Each text line occupies $6\,U$ ($= 1.5 \times$ fontSize), which corresponds to the 1.5 line-height ratio adopted to satisfy the WCAG 2.1 resilience criterion [15] and validated by readability research [16].

The term $2 \cdot d \cdot w \cdot U$ represents the total vertical padding (top and bottom combined). It grows linearly with both density d and wrapping level w.

1) Single-Line Controls: For the most common case — a single-line bounded control ($n = 1$, $w = 1$):

$$h = (6 + 2d)\, U. \tag{8}$$

At the default density $d = 1.5$:

$$h = 9\, U. \tag{9}$$

At fontSize $= 16\,\text{px}$ ($U = 4\,\text{px}$), this yields $h = 36\,\text{px}$.

2) Multi-Line Blocks: For multi-line bounded blocks ($w = 2$):

$$h = (6n + 6)\, U \quad \text{at } d = 1.5. \tag{10}$$

This adds one full text-line height ($6\,U$) of total vertical padding, split equally above and below the content region.

3) Height Reference: Table I shows the resulting heights across all wrapping levels at the default density.

TABLE I
ELEMENT HEIGHT AT DEFAULT DENSITY $d = 1.5$, $n = 1$.

w	Formula	Height (U)	Height (px)
0	$6\,U$	6	24
1	$9\,U$	9	36
2	$12\,U$	12	48
3	$15\,U$	15	60

D. Padding and Radius

1) Block Padding: Vertical (block-axis) padding follows directly from the height formula:

$$p_{\text{block}} = d \cdot w \cdot U. \tag{11}$$

At $w = 0$, block padding is zero. At $w = 1$ with default density, $p_{\text{block}} = 1.5\,U = 6\,\text{px}$.

2) Inline Padding: Horizontal (inline-axis) padding uses a width multiplier derived from the wrapping level:

$$p_{\text{inline}} = \begin{cases} 0 & w = 0, \text{ unbounded inline} \\ 2 \cdot d \cdot U & w = 0, \text{ bounded inline} \\ \lceil 3/w \rceil \cdot d \cdot w \cdot U & w \geq 1 \end{cases} \tag{12}$$

The $\lceil 3/w \rceil$ factor produces the following multipliers:

w	$\lceil 3/w \rceil$	p_{inline} at $d = 1.5$
1	3	$4.5\,U$
2	2	$6\,U$
3	1	$4.5\,U$

This non-monotonic behavior is intentional: single-line controls ($w = 1$) and structural sections ($w = 3$) share the same inline padding, while multi-line blocks ($w = 2$) receive wider padding to accommodate wider content.

For bounded inline elements at $w = 0$ (e.g., tag, badge, inline code), inline padding is $2dU = 3\,U$ at default density. Unbounded inline content (plain text, icon) receives no padding.

3) Border Radius: Border radius is defined to equal the block padding:

$$r = p_{\text{block}} = d \cdot w \cdot U. \tag{13}$$

This identity ensures that the corner curvature is always proportional to the surrounding whitespace, maintaining consistent visual weight across density levels.

4) Spacing Reference: Table II shows all spatial properties at default density.

TABLE II
SPATIAL PROPERTIES AT DEFAULT DENSITY $d = 1.5$, $n = 1$. VALUES IN U (PX AT 16 PX FONT SIZE).

Property	$w = 0$	$w = 1$	$w = 2$	$w = 3$
Height	6 (24)	9 (36)	12 (48)	15 (60)
p_{block}	0 (0)	1.5 (6)	3 (12)	4.5 (18)
p_{inline}	3 (12)*	4.5 (18)	6 (24)	4.5 (18)
Radius	0 (0)	1.5 (6)	3 (12)	4.5 (18)

* Bounded inline only (tag, badge, code). Unbounded inline has zero padding.

E. Sub-Baseline Scale

Elements intentionally sized below the $6\,U$ text baseline — such as checkboxes, radio buttons, range sliders, and progress bars — follow a fixed proportional sub-scale:

$$\{2\,U,\ 4\,U,\ 6\,U\}. \tag{14}$$

These values remain constant across density levels. The density factor d does not affect sub-baseline elements because they have no wrapping-level padding to scale; their geometry is determined entirely by the base unit U.

The sub-scale provides three discrete heights:

Height	px at 16 px	Examples
$2\,U$	8 px	progress bar, popover arrow
$4\,U$	16 px	checkbox, radio, range thumb, switch
$6\,U$	24 px	text line (baseline)

The boundary at $6\,U$ marks the transition: elements at or above this height enter the parametric model governed by Equation 7; elements below it are governed by the sub-scale.

F. Spacing Emission

The geometry formulas produce numeric values in units of U. The final step converts these to CSS lengths via a spacing function:

$$\texttt{themeSpacing}(n) = \frac{n}{4}\,\text{em}. \tag{15}$$

This definition has a critical consequence: all emitted spacing is *relative to the local font size*, not to a fixed pixel value. Because $U = \text{fontSize}/4$ and $1\,\text{em} = \text{fontSize}$, the conversion $n \mapsto n/4\,\text{em}$ preserves the proportional relationship established by the geometry formulas.

Consider a button at default density ($d = 1.5$, $w = 1$):

$$p_{\text{block}} = d \cdot w \cdot U = 1.5\,U$$
$$\rightarrow \texttt{themeSpacing}(1.5) = 0.375\,\text{em}.$$

If the subtree inherits a larger font size (e.g., 20 px instead of 16 px), the em-based value automatically scales: $0.375 \times 20 = 7.5$ px instead of $0.375 \times 16 = 6$ px. No formula change is required.

This property makes the model *resolution-independent*: the same parametric expressions produce correct geometry at any font size, any display density, and any responsive breakpoint.

G. Context Inheritance

Both size and density propagate through the element tree via inherited data attributes (`dataSize` and `dataDensity`). When a container sets a density attribute, every descendant element that reads `themeDensity()` receives the updated value without explicit prop passing.

This inheritance model has three properties:

1) **Local override:** A container can set `dataDensity = "increase-1"` to compact all children by one density step, while the rest of the page remains unchanged.
2) **Relative adjustment:** Offsets such as `"increase-N"` and `"decrease-N"` are resolved relative to the inherited context, not to an absolute index. This allows nested overrides to compose: a compact sidebar inside a comfortable page produces a doubly compact section.
3) **Reactive propagation:** Each context lookup registers a listener on the ancestor node. When the attribute changes at runtime, all descendants re-evaluate their geometry in a single microtask batch, producing consistent visual updates without layout thrashing.

The same inheritance mechanism applies to the type scale (`dataSize`) and to tone context (`dataTone`). This shared propagation model means that size, density, and color context are resolved through one unified traversal, keeping the runtime cost constant regardless of the number of themed properties.

H. Orthogonal Axes

The sizing model is one of three independent axes in the design system:

1) **Size** (type scale): Controls the local font size and therefore the base unit U. Changing the size index rescales all geometry proportionally.
2) **Density** (spatial compactness): Controls the padding multiplier d. Changing density adjusts whitespace without affecting font size or color.
3) **Tone** (color context): Controls surface lightness and semantic color. Changing tone has no effect on spatial geometry.

These axes are algebraically independent: no formula for one axis references a variable from another. Size enters geometry only through U; density enters only through d; tone enters only through the color resolution pipeline.

This orthogonality has a practical consequence: any combination of size, density, and tone is valid. A compact ($d = 0.75$), large-text (`dataSize = "increase-2"`), dark-surface (`dataTone = "shift-17"`) configuration produces correct geometry and correct contrast without special-case handling.

All three axes share the same context inheritance mechanism (Section V-G), but operate on different variables. This structural parallel — independent parametric models over a shared propagation substrate — keeps each axis simple while allowing arbitrary composition.

I. Layout Spacing

The geometry formulas govern *internal* element spacing: padding, height, and radius within a single element. Spacing *between* separate elements (gaps, margins) is not formula-driven; it follows a practical guideline:

- Horizontal gap or margin-inline should be at least the related p_{inline}.
- Vertical gap or margin-block should be at least the related p_{block}.

At default density ($d = 1.5$), this produces:

Direction	Minimum gap	px at 16 px
Horizontal ($w = 1$)	$4.5\,U$	18
Vertical ($w = 2$)	$3\,U$	12

This distinction is intentional. Internal geometry is constrained by structural proportion (content must fit inside its container), so it benefits from closed-form expressions. Layout spacing is constrained by content flow and visual rhythm, which depend on context that the formula cannot anticipate. The guideline provides a lower bound that maintains visual consistency without over-constraining layout decisions.

VI. Coverage Proof

To validate that the sizing model provides complete coverage, every production *patch* — a composable function that applies geometry, behavior, and styling to a DOM element — in the UI library is mapped to its height family. The classification confirms that each patch maps to exactly one row of the sizing taxonomy, with no exceptions.

A. Height Families

Patches are grouped by their height expression into ten families:

1) **Separator** (1 px): A degenerate fixed-height case.
2) **Sub-baseline** $2\,U$: Elements below the text baseline, governed by the sub-scale (Section V-E).
3) **Sub-baseline** $4\,U$: Controls below the text baseline (Section V-E).
4) **Text-line** $6\,U$ ($n = 1$, $w = 0$): Inline elements whose height equals one text line.
5) **Flex list** $6n\,U$ ($w = 0$): Inline containers whose height scales with content lines.

6) **Single-line control** $9\,U$ ($n = 1$, $w = 1$, $d = 1.5$): Bounded controls with one text line.
7) **Table row** $9n\,U$ ($w = 1$, $d = 1.5$): Rows scaling with line count.
8) **Multi-line block** $(6n+6)\,U$ ($w = 2$, $d = 1.5$): Bounded blocks with vertical padding equal to one text line.
9) **Layout region** (n/a): Content-determined height.
10) **Behavior** (n/a): No visual height contribution.

B. Complete Mapping

Table III lists all 69 production patches with their assigned parameters and predicted heights. Every formula-governed patch (54 of 69) matches the prediction exactly. The remaining patches are classified as sub-baseline (6), separator (1), layout region (5), or behavior (3), all of which are accounted for by the model's explicit boundary categories. Note that sub-baseline and separator patches use fixed heights within the model; layout and behavior patches are explicitly outside the parametric formula's scope.

The coverage rate is $69/69 = 100\%$. No patch requires a height value outside the model, and no patch maps to more than one family.

VII. Industry Benchmark

To evaluate the sizing model against production practice, this section benchmarks single-line control heights across ten major design systems. The goal is not to show that the formula was fitted to these systems, but to demonstrate that independently developed systems converge on the same discrete height values that the formula produces.

A. Method

For each design system, the default button height and all named size variants (e.g., small, medium, large) were extracted from source code or published design tokens. Table V reports the results. Table IV maps each density value to its formula output and matching design system variants.

The formula predicts single-line control height as:

$$h = (6 + 2d) \cdot U, \quad n = 1,\ w = 1. \tag{16}$$

At fontSize $= 16\,\text{px}$ ($U = 4\,\text{px}$), integer density values produce:

$$d = 0 \to 24, \quad d = 1 \to 32, \quad d = 2 \to 40, \quad d = 3 \to 48\,\text{px}.$$

B. Results

Table VI summarizes the convergence. The height $32\,\text{px}$ appears as a default or named variant in 8 of 10 systems (80%). The height $40\,\text{px}$ appears in 6 of 10 (60%). Three of the four formula-predicted heights $(24, 32, 40)$ appear in at least 60% of systems; the fourth (48) appears in 40%.

C. Analysis

Three observations emerge from the benchmark:

1) **The $4\,\text{px}$ grid is universal.** Every surveyed system uses heights that are multiples of $4\,\text{px}$, except MUI, whose fractional heights are a known issue [17]. The base unit $U = \text{fontSize}/4$ aligns with this grid by construction.
2) **$32\,\text{px}$ is the industry default.** Eight of ten systems include $32\,\text{px}$ as either the default or medium button height. In the formula, this corresponds to $d = 1$ at $U = 4\,\text{px}$: the lowest integer density above zero. This suggests that $d = 1$ captures the minimum viable padding for a bounded control.
3) **Size variants follow the formula's density steps.** The progression $24 \to 32 \to 40 \to 48$ (step size $8\,\text{px} = 2\,U$) matches the formula at $d = 0, 1, 2, 3$. Each density increment adds $2\,U = 8\,\text{px}$ of height (one unit of block padding on each side). Systems that use finer gradations (e.g., Primer's $28\,\text{px}$) correspond to half-integer density values ($d = 0.5$), which the formula supports algebraically. Heights below $24\,\text{px}$ (e.g., Ant Design's $16\,\text{px}$, Polaris's $20\,\text{px}$) fall outside the model's valid density range ($d \geq 0$) for $w = 1$ and are better classified as sub-baseline elements.

D. Deviation Analysis

The only systematic deviation is MUI, whose button heights ($\approx 31, 37, 42\,\text{px}$) are fractional. This occurs because MUI uses fontSize $= 14\,\text{px}$ with lineHeight $= 1.75$, producing a content height of $14 \times 1.75 = 24.5\,\text{px}$. The fractional base breaks alignment with the $4\,\text{px}$ grid. At fontSize $= 14\,\text{px}$, $U = 3.5\,\text{px}$, and $h = (6+2d) \times 3.5$, which does not produce integer values at integer d. This confirms a prediction of the model: the $4\,\text{px}$ grid alignment requires fontSize to be a multiple of 4.

All other systems that use fontSize $= 14\,\text{px}$ avoid this issue by decoupling control height from line-height: they specify explicit height tokens (e.g., Ant Design's `controlHeight = 32`) rather than deriving height from padding plus content. The formula's approach — deriving height from U — produces correct grid-aligned results when fontSize is a multiple of 4.

E. Cross-System Consistency

The benchmark reveals that the sizing model is not an invention but a formalization. The formula $h = (6+2d) \cdot U$ does not prescribe heights; it describes the structural relationship that the industry has independently converged on. The contribution is making this relationship explicit, parametric, and derivable from constraints (Section IV).

F. Limitations

The sizing model governs internal element geometry. Several categories of UI concern fall outside its scope:

- **Layout regions.** Structural containers such as dialogs, drawers, and menus have content-determined height. The model assigns them wrapping level $w = 3$ for padding and radius computation, but does not constrain their

overall height. This is intentional: layout regions frame arbitrary nested content whose extent cannot be predicted by a closed-form expression.

- **Behavior patches.** Patches that apply logic without contributing visual geometry (e.g., form validation, field binding, transition orchestration) are outside the spatial model. They occupy no height and require no sizing parameters.
- **Separator.** The horizontal rule uses a fixed 1 px height. This is a degenerate case that does not participate in the U-based model because a separator's role is to mark a boundary, not to contain content.
- **Custom artistic elements.** The model provides a baseline geometry. Elements with deliberately non-standard proportions (e.g., hero banners, decorative illustrations) may override the formula. The model does not prevent overrides; it establishes the default from which overrides are measured.
- **CSS border interaction.** The height formula assumes the CSS box model without borders. A 1 px border adds 2 px to rendered height (5.56% deviation at $w = 1$, $d = 1.5$). The model recommends `outline` or `box-shadow` for visual edges, but does not enforce this [23].

These boundaries are explicit. The coverage proof (Table III) accounts for all five categories: layout regions and behavior patches are classified as n/a, the separator is classified as a fixed-height exception, and all remaining patches (54 of 69) are fully governed by the parametric formula with zero deviations.

VIII. Discussion

The sizing model presented in this paper addresses only one half of the visual structure of a user interface: *where things are and how large they are*. The other half — *what color they are and what that color means* — is addressed by the Chromametry framework [29], which defines quantitative metrics for sequential monochromatic color palettes and derives the contrast span K as a fixed index rule for WCAG-compliant color selection.

Together, these two models form a complete structural description of UI:

- **Sizing model** (this paper): Governs height, padding, radius, and spacing through (n, w, d) and the base unit U. Every spatial property is a closed-form expression.
- **Tone model** (Chromametry): Governs surface color, text color, stroke color, and interaction feedback through an 18-step lightness ramp with contrast span $K = 9$. Every color role is a fixed index offset from the surface anchor.

The two models are algebraically independent — no sizing formula references a tone variable, and no tone formula references a spatial variable — yet they share the same propagation mechanism: context inheritance through the element tree via data attributes (`dataSize`, `dataDensity`, `dataTone`). This means a single container attribute change can uniformly adjust either the spatial geometry or the color context of all descendant elements.

This structural parallel is not accidental. A UI element has exactly two perceptual dimensions that a design system must govern:

1) **Spatial structure:** How much space the element occupies, how much whitespace surrounds its content, and how its boundary is shaped. These are the concerns of the sizing model.
2) **Chromatic structure:** What lightness the surface has, what contrast the text achieves, and how interaction states are distinguished. These are the concerns of the tone model.

Existing design systems address both dimensions, but through unrelated mechanisms: spatial tokens for sizing, color tokens for palette. The two token sets are maintained independently, with no shared algebra connecting them. The result is that changing density requires updating spatial tokens while leaving color tokens unchanged (correct), but there is no structural guarantee that the two systems remain compatible — only convention.

The combination of the sizing model and the Chromametry tone model replaces this convention with a structural guarantee. Because both models are parametric (expressions over a small variable set) rather than enumerative (flat token tables), they can be composed, overridden, and propagated through the same inheritance mechanism. An element's full visual specification reduces to five variables:

Variable	Axis	Source
n	Sizing	Intrinsic (patch type)
w	Sizing	Intrinsic (patch type)
d	Sizing	Inherited (`dataDensity`)
U	Sizing	Inherited (`dataSize` $\rightarrow$ fontSize)
T	Tone	Inherited (`dataTone`)

Of these, n and w are fixed per patch type. The remaining three (d, U, T) are inherited from context. This means the entire visual appearance of any element in the library is determined by three inherited context values and two compile-time constants — with no per-element overrides, no magic numbers, and no token tables.

The sizing model addresses the spatial half of UI structure; the Chromametry tone model addresses the chromatic half. Neither is sufficient alone. Together, they suggest that a quantitative, parametric approach can cover the two primary perceptual dimensions of interface design.

IX. Conclusion

This paper has presented a unified sizing model for user interfaces. The model derives all element geometry from three variables — intrinsic text lines n, wrapping level w, and density factor d — expressed in a base unit $U = \text{fontSize}/4$.

The key contributions are:

1) A closed-form height formula $h = (n \cdot 6 + 2 \cdot d \cdot w) \cdot U$ that covers all height-defined elements.

2) A four-level wrapping taxonomy that classifies every UI element by boundary semantics, grounded in Gestalt proximity principles [11], [12].
3) A five-step density scale whose default value reproduces the canonical button heights adopted by MUI, Ant Design, Chakra UI, GitHub Primer, and Adobe Spectrum [17], [18], [19], [20], [21].
4) Em-based spacing emission that makes the model resolution-independent: the same formulas produce correct geometry at any font size without modification.
5) Context inheritance through the element tree, enabling local overrides that compose without global coordination.
6) Orthogonality with the tone (color) axis: size, density, and tone operate as three independent parametric models over a shared propagation substrate, allowing arbitrary composition without special-case handling.
7) A sub-baseline scale for elements below the text-line height, and a dimension table confirming that 69 production patches map to the model without exceptions.

By replacing per-element token tables with parametric expressions, the model ensures that density changes, font-size rescaling, and responsive adaptation propagate uniformly across the entire UI library. The underlying principle is that spatial design should be derived, not declared.

References

[1] R. Bringhurst. *The Elements of Typographic Style*, Version 4.0. Hartley & Marks, 2012.

[2] T. Brown. More meaningful typography. *A List Apart*, May 2011. Available at https://alistapart.com/article/more-meaningful-typography/.

[3] N. Curtis. Space in design systems. *EightShapes*, September 2016. Available at https://medium.com/eightshapes-llc/space-in-design-systems-188bcbae0d62.

[4] A. Kutschmann. Design token-based UI architecture. Published on *martinfowler.com*, December 2024. Available at https://martinfowler.com/articles/design-token-based-ui-architecture.html.

[5] Design Tokens Community Group. *Design Tokens Format Module 2025.10*. W3C Community Group Report, 2025. Available at https://www.designtokens.org/tr/drafts/format/.

[6] E. Dahl. Intro to the 8-point grid system. *Built to Adapt*, 2017. Available at https://medium.com/built-to-adapt/intro-to-the-8-point-grid-system-d2573cde8632.

[7] Google. Applying density. *Material Design 2 Guidelines*, 2019. Available at https://m2.material.io/design/layout/applying-density.html.

[8] U. Kravets. Using Material density on the web. *Google Design*, 2020. Available at https://medium.com/google-design/using-material-density-on-the-web-59d85f1918f0.

[9] Y. Gabillon, S. Lepreux, and K. M. de Oliveira. Towards ergonomic user interface composition: A study about information density criterion. In *Human-Computer Interaction (HCI 2013)*, LNCS 8004, pages 216–225. Springer, 2013.

[10] E. R. Tufte. *The Visual Display of Quantitative Information*. Graphics Press, 2nd edition, 2001.

[11] M. Wertheimer. Untersuchungen zur Lehre von der Gestalt, II. *Psychologische Forschung*, 4:301–350, 1923. English translation in W. Ellis, *A Source Book of Gestalt Psychology*, Routledge, 1938.

[12] J. Wagemans, J. H. Elder, M. Kubovy, S. E. Palmer, M. A. Peterson, M. Singh, and R. von der Heydt. A century of Gestalt psychology in visual perception: I. Perceptual grouping and figure–ground organization. *Psychological Bulletin*, 138(6):1172–1217, 2012.

[13] J. Nielsen. Enhancing the explanatory power of usability heuristics. In *CHI '94: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 152–158. ACM, 1994.

[14] B. Shneiderman, C. Plaisant, M. Cohen, S. Jacobs, and N. Elmqvist. *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. Pearson, 6th edition, 2016.

[15] W3C. *Web Content Accessibility Guidelines (WCAG) 2.1*. 2018. Success Criterion 1.4.12: Text Spacing. Available at https://www.w3.org/TR/WCAG21/#text-spacing.

[16] L. Rello, M. Pielot, and M.-C. Marcos. Make it big! The effect of font size and line spacing on online readability. In *CHI '16: Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, pages 3637–3648. ACM, 2016.

[17] MUI. *Material UI*. 2024. Available at https://mui.com/.

[18] Ant Group. *Ant Design*. 2024. Available at https://ant.design/.

[19] S. Adebayo. *Chakra UI*. 2024. Available at https://chakra-ui.com/.

[20] GitHub. *Primer Design System*. 2024. Available at https://primer.style/.

[21] Adobe. *Spectrum Design System*. 2024. Available at https://spectrum.adobe.com/.

[22] Google. *Material Design 3*. 2023. Available at https://m3.material.io/.

[23] W3C. *CSS Box Model Module Level 3*. 2023. Available at https://www.w3.org/TR/css-box-3/.

[24] Microsoft. *Fluent UI*. 2024. Available at https://fluent2.microsoft.design/.

[25] IBM. *Carbon Design System*. 2024. Available at https://carbondesignsystem.com/.

[26] Atlassian. *Atlassian Design System*. 2024. Available at https://atlassian.design/.

[27] Shopify. *Polaris Design System*. 2024. Available at https://polaris.shopify.com/.

[28] Salesforce. *Lightning Design System*. 2024. Available at https://www.lightningdesignsystem.com/.

[29] H. K. Nguyen. Web-accessible color palette metrics. 2025. Available at https://github.com/chromametry/chromametry.

TABLE III

COMPLETE COVERAGE PROOF: EVERY PRODUCTION PATCH MAPPED TO ITS HEIGHT FAMILY. PARAMETERS AT DEFAULT DENSITY $d = 1.5$, FONTSIZE $= 16$ PX ($U = 4$ PX). A CHECKMARK (✓) INDICATES THE PATCH'S IMPLEMENTED HEIGHT MATCHES THE FORMULA PREDICTION EXACTLY.

Family	Patch	n	w	d	Formula	Height	Match
Separator							
	Horizontal Rule	—	—	—	fixed	1 px	✓
Sub-baseline ($2\,U$)							
	Progress	—	—	—	$2\,U$	8 px	✓
	Popover Arrow	—	—	—	$2\,U$	8 px	✓
Sub-baseline ($4\,U$)							
	Input Checkbox	—	—	—	$4\,U$	16 px	✓
	Input Radio	—	—	—	$4\,U$	16 px	✓
	Input Range	—	—	—	$4\,U$	16 px	✓
	Input Switch	—	—	—	$4\,U$	16 px	✓
Text-line ($6\,U$, i.e. n=1, w=0)							
	Abbreviation	1	0	—	$6\,U$	24 px	✓
	Badge	1	0	—	$6\,U$	24 px	✓
	Breadcrumb Ell.	1	0	—	$6\,U$	24 px	✓
	Button Switch	1	0	—	$6\,U$	24 px	✓
	Code	1	0	—	$6\,U$	24 px	✓
	Divider	1	0	—	$6\,U$	24 px	✓
	Emphasis	1	0	—	$6\,U$	24 px	✓
	Heading	1	0	—	$6\,U$	24 px	✓
	Icon	1	0	—	$6\,U$	24 px	✓
	Keyboard	1	0	—	$6\,U$	24 px	✓
	Label	1	0	—	$6\,U$	24 px	✓
	Link	1	0	—	$6\,U$	24 px	✓
	Mark	1	0	—	$6\,U$	24 px	✓
	Skeleton	1	0	—	$6\,U$	24 px	✓
	Small	1	0	—	$6\,U$	24 px	✓
	Spinner	1	0	—	$6\,U$	24 px	✓
	Strong	1	0	—	$6\,U$	24 px	✓
	Subscript	1	0	—	$6\,U$	24 px	✓
	Superscript	1	0	—	$6\,U$	24 px	✓
	Tag	1	0	—	$6\,U$	24 px	✓
	Toggle	1	0	—	$6\,U$	24 px	✓
Flex list ($6n\,U$)							
	Breadcrumb	n	0	—	$6n\,U$	$24n$ px	✓
	Description List	n	0	—	$6n\,U$	$24n$ px	✓
	Ordered List	n	0	—	$6n\,U$	$24n$ px	✓
	Paragraph	n	0	—	$6n\,U$	$24n$ px	✓
	Unordered List	n	0	—	$6n\,U$	$24n$ px	✓
Single-line control ($9\,U$, i.e. n=1, w=1, d=1.5)							
	Avatar	1	1	1.5	$9\,U$	36 px	✓
	Button	1	1	1.5	$9\,U$	36 px	✓
	Combobox	1	1	1.5	$9\,U$	36 px	✓
	Input Color	1	1	1.5	$9\,U$	36 px	✓
	Input Date Time	1	1	1.5	$9\,U$	36 px	✓
	Input File	1	1	1.5	$9\,U$	36 px	✓
	Input Number	1	1	1.5	$9\,U$	36 px	✓
	Input Search	1	1	1.5	$9\,U$	36 px	✓
	Input Text	1	1	1.5	$9\,U$	36 px	✓
	Menu Item	1	1	1.5	$9\,U$	36 px	✓
	Pagination	1	1	1.5	$9\,U$	36 px	✓
	Select	1	1	1.5	$9\,U$	36 px	✓
	Select Box	1	1	1.5	$9\,U$	36 px	✓
	Select Item	1	1	1.5	$9\,U$	36 px	✓
	Select List	1	1	1.5	$9\,U$	36 px	✓
	Tab	1	1	1.5	$9\,U$	36 px	✓
	Tooltip	1	1	1.5	$9\,U$	36 px	✓
Table row ($9n\,U$)							
	Table	n	1	1.5	$9n\,U$	$36n$ px	✓
Multi-line block ($(6n{+}6)\,U$, i.e. w=2, d=1.5)							
	Alert	n	2	1.5	$(6n{+}6)\,U$	$(24n{+}24)$ px	✓
	Blockquote	n	2	1.5	$(6n{+}6)\,U$	$(24n{+}24)$ px	✓
	Details	n	2	1.5	$(6n{+}6)\,U$	$(24n{+}24)$ px	✓
	Figure	n	2	1.5	$(6n{+}6)\,U$	$(24n{+}24)$ px	✓
	Image	n	2	1.5	$(6n{+}6)\,U$	$(24n{+}24)$ px	✓
	Popover	n	2	1.5	$(6n{+}6)\,U$	$(24n{+}24)$ px	✓

TABLE IV
FORMULA OUTPUT $h = (6 + 2d) \cdot U$ AT $U = 4$ PX AND MATCHING DESIGN SYSTEM VARIANTS.

d	$(6 + 2d) \cdot U$	Height	Industry matches
0	$(6 + 0) \times 4$	24 px	Ant XS, Chakra 2xs, Primer XS
1	$(6 + 2) \times 4$	32 px	Ant md, Primer md, Fluent md
1.5	$(6 + 3) \times 4$	36 px	Chakra sm, Polaris lg
2	$(6 + 4) \times 4$	40 px	Ant lg, Chakra md, Primer lg
3	$(6 + 6) \times 4$	48 px	Chakra xl, Carbon lg, Primer XL

TABLE V

DEFAULT SINGLE-LINE CONTROL HEIGHTS ACROSS TEN DESIGN SYSTEMS. ALL VALUES IN PIXELS. HEIGHTS MARKED WITH * ARE COMPUTED FROM PADDING + LINE-HEIGHT (NOT EXPLICIT TOKENS). THE FORMULA COLUMN SHOWS $h = (6 + 2d) \cdot U$ AT $U = 4$ PX (FONTSIZE $= 16$ PX).

System	XS/Compact	Small	Medium	Large	XL	Base font	Source
MUI	—	31*	**37***	42*	—	14 px	GitHub
Ant Design	16	24	**32**	40	—	14 px	seed.ts
Chakra UI v3	24	36	**40**	44	48	16 px	recipe
GitHub Primer	24	28	**32**	40	48	14 px	primitives
Adobe Spectrum	20	24	**32**	40	48	14 px	tokens
Fluent UI	—	24	**32**	40	—	14 px	styles
Carbon (IBM)	24	32	**40**	48	64	14 px	layout
Atlassian	—	24	**32**	—	—	14 px	tokens
Shopify Polaris	—	20	**32**	36	—	16 px	CSS
Salesforce SLDS	—	—	**32**	—	44	13 px	SCSS
Formula at $U = 4$:		d=0: 24	d=1: **32**	d=2: 40	d=3: 48		

MUI heights are fractional because MUI uses lineHeight $= 1.75$ at fontSize $= 14$ px, producing non-integer content heights. Values shown are rounded.

TABLE VI

CONVERGENCE FREQUENCY: HOW MANY OF THE TEN SURVEYED DESIGN SYSTEMS USE EACH HEIGHT AS A DEFAULT OR NAMED SIZE VARIANT FOR SINGLE-LINE CONTROLS.

Height (px)	d at U=4	Systems	Fraction
24	0	6/10	60%
32	1	8/10	80%
40	2	6/10	60%
48	3	4/10	40%